

SmartFetch

Secure Adaptive Web Data Extraction Platform

Professor: Manel Abdelkader

Youssef Mabrouk | Adam Hajri | Jaouher Chouchane | Mohammed Sanad Boukhmiss

Project Context

SmartFetch is a web-based data extraction platform designed to help users collect structured information from public webpages in a controlled and secure way.

Why This Project?

- ▶ Web data is often useful but difficult to extract manually.
- ▶ Traditional scraping requires technical knowledge.
- ▶ Unsafe scraping can expose the backend to security risks.

Project Direction

- ▶ Provide a simple extraction interface.
- ▶ Keep fetching and parsing controlled by the backend.
- ▶ Apply security checks before processing external URLs.

Presentation Structure

Engineering View

- ① Problem and objectives
- ② Functional workflow
- ③ System architecture
- ④ Backend and API design

Security View

- ⑤ Threat model
- ⑥ Security controls
- ⑦ Testing and validation
- ⑧ Limitations and improvements

Defense Position

This presentation explains SmartFetch not only as a working web application, but as a security-aware software architecture with explicit trust boundaries and defensive controls.

Problem Statement

Core Problem

Users need structured data from public webpages, but traditional web scraping requires programming knowledge, manual selector writing, and careful handling of unsafe external requests.

User-side difficulties

- ▶ Manual copy-paste is slow and error-prone.
- ▶ Non-technical users do not know HTML selectors.
- ▶ Exporting clean CSV/JSON usually requires extra tools.

Security-side difficulties

- ▶ External URLs are untrusted input.
- ▶ A backend fetcher can be abused for SSRF.
- ▶ Redirects, private IPs, and large responses must be controlled.

Security interpretation: SmartFetch is a controlled gateway between the user, the backend service, and untrusted external web content.

Proposed Solution

SmartFetch Solution

SmartFetch provides a secure web interface where users submit a public webpage URL, select the type of data they want to extract, preview the results, and export the cleaned output as JSON or CSV.

User-Side Solution

- ▶ Simple URL-based extraction.
- ▶ No need to write code or CSS selectors manually.
- ▶ Preview extracted results before export.
- ▶ Export data in structured formats.

Security-Side Solution

- ▶ Backend validates the URL before fetching.
- ▶ Private and localhost addresses are blocked.
- ▶ Redirects and response types are controlled.
- ▶ Only safe HTML content is processed.

Project Objectives

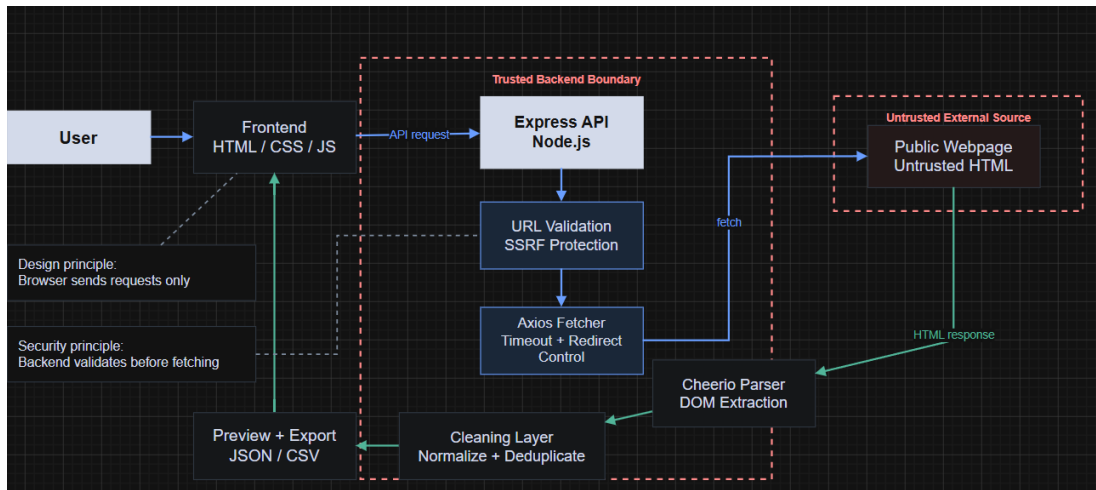
Functional Objectives

- ▶ Analyze one public webpage at a time.
- ▶ Detect available extraction categories.
- ▶ Extract headings, paragraphs, links, images, tables, lists, metadata, prices, and dates.
- ▶ Support grouped card-style extraction.
- ▶ Export results as JSON and CSV.

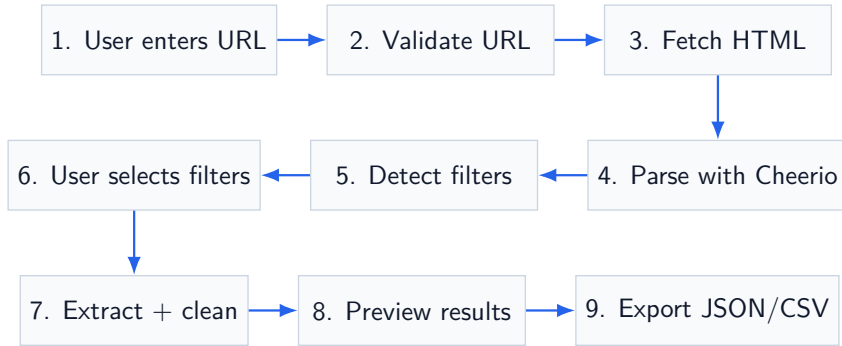
Security Objectives

- ▶ Validate every submitted URL.
- ▶ Allow only HTTP and HTTPS schemes.
- ▶ Block localhost and private network targets.
- ▶ Re-validate each redirect destination.
- ▶ Limit request time, redirect count, and response size.

High-Level Architecture



Request Processing Workflow



Main Point

The workflow is deliberately sequential: validation happens before fetching, and cleaning happens before preview or export.

Technology Stack and Responsibility Separation

Layer	Technology	Responsibility
Frontend	HTML, CSS, JavaScript	User input, filter rendering, preview display, browser-side download.
API Layer	Node.js, Express	Defines routes, validates requests, coordinates fetching and extraction.
Fetching Layer	Axios	Downloads external HTML with timeout, redirect control, and size limits.
Parsing Layer	Cheerio	Loads HTML and performs DOM-style extraction.
Security Layer	URL parser, DNS lookup, IP checks	Blocks unsafe targets and revalidates redirects.
Test Layer	Node test runner	Verifies extraction, grouping, CSV output, and private IP detection.

Threat Model: What Must Be Protected?

Assets

- ▶ Backend server and local network.
- ▶ Application availability.
- ▶ Integrity of extracted/exported data.
- ▶ User trust in previewed results.

Main Threats

- ▶ Server-Side Request Forgery (SSRF).
- ▶ Redirects to internal addresses.
- ▶ Very large or slow responses.
- ▶ Malformed URLs and unsupported schemes.
- ▶ Noisy or misleading extracted data.

Trust Boundary

The most important boundary is between the trusted Express server and the untrusted URL supplied by the user.

Security Controls Implemented

Risk	Control in SmartFetch	Security Value
SSRF to localhost	Blocks localhost, 127.0.0.1, ::1, and private IP ranges.	Protects host and local services.
DNS-based internal targets	Resolves hostname with DNS lookup and checks returned IP addresses.	Blocks domains resolving to private networks.
Dangerous schemes	Allows only <code>http:</code> and <code>https:</code> .	Prevents file, ftp, and unsupported schemes.
Redirect abuse	Disables automatic redirects and validates each redirect manually.	Prevents redirect-to-internal bypasses.
Slow or large responses	Uses request timeout and maximum HTML byte size.	Improves availability.
Non-HTML targets	Checks content type before parsing.	Avoids processing arbitrary files.

How Security Controls Were Implemented

1. Blocking Unsafe URLs

- ▶ The backend parses every submitted URL before sending any request.
- ▶ Only `http://` and `https://` URLs are accepted.
- ▶ Dangerous schemes such as `file://`, `ftp://`, and malformed URLs are rejected.
- ▶ URLs containing credentials are blocked to avoid unsafe hidden authentication patterns.

2. Blocking Private and Local Targets

- ▶ Localhost targets such as `localhost`, `127.0.0.1`, and `::1` are rejected.
- ▶ Private IP ranges such as `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16` are blocked.
- ▶ The backend performs DNS lookup to detect domains that resolve to private IP addresses.

3. Blocking Non-HTML Responses

- ▶ After fetching, the backend checks the response `Content-Type`.
- ▶ Only HTML responses are processed by the extraction engine.
- ▶ Non-HTML content such as PDFs, images, executables, archives, or binary files is rejected.

4. Redirect and Timeout Protection

- ▶ Redirects are not trusted automatically.
- ▶ Each redirect destination is validated again before continuing.
- ▶ Request timeout prevents the server from waiting indefinitely for slow websites.
- ▶ Maximum response-size control prevents the backend from processing very large pages.

URL validation

- ▶ Parse URL safely.
- ▶ Reject credentials in URL.
- ▶ Reject localhost hostnames.
- ▶ Reject private IPv4 and IPv6 ranges.
- ▶ Resolve hostnames and validate DNS results.

Only allowed:

`http://...`

`https://...`

Blocked examples:

`file:///etc/passwd`

`http://localhost:3000`

`http://127.0.0.1`

`http://192.168.1.1`

`http://10.0.0.5`

`URL with username/password`

Defense Explanation

The backend treats the URL as hostile input. It is validated before the first request and again after every redirect.

POST /api/analyze

- ▶ Receives the URL.
- ▶ Validates and fetches HTML.
- ▶ Detects available filters.
- ▶ Returns page title, redirect info, and filter counts.

POST /api/extract

- ▶ Receives URL and selected filters.
- ▶ Uses cached HTML when available.
- ▶ Extracts separate or grouped data.
- ▶ Returns sections, summary, and CSV output.

Design Quality

The API separates analysis from extraction. This improves usability because the user sees available filters before requesting final extraction.

Extraction Engine

Standard filters

- ▶ `extractHeadings()`
- ▶ `extractParagraphs()`
- ▶ `extractLinks()`
- ▶ `extractImages()`
- ▶ `extractTables()`
- ▶ `extractLists()`

Advanced filters

- ▶ `extractMetadata()`
- ▶ `extractPrices()`
- ▶ `extractDates()`
- ▶ `extractGrouped()`
- ▶ `buildCsv()`
- ▶ `uniqueRows()`

Why This Is Professionally Structured

Each extraction responsibility is isolated in a function. This improves testing, maintainability, debugging, and future extension.

Cleaning Operations

- ▶ Normalize whitespace.
- ▶ Remove empty values.
- ▶ Deduplicate rows.
- ▶ Convert relative URLs to absolute URLs.
- ▶ Remove noisy links such as `javascript:`, `mailto:`, `tel:`, and anchors.

Output Benefits

- ▶ Cleaner preview.
- ▶ Less duplicated exported data.
- ▶ More reliable CSV columns.
- ▶ Better link usability after export.
- ▶ Easier interpretation by the user.

Testing Strategy

Testing Focus

The tests focus on the high-risk and high-value parts of the system: parsing, cleaning, grouping, export formatting, and private IP protection.

Test Area	Validated Behavior	Status
Text cleaning	Whitespace normalization and duplicate removal.	✓
Extraction	Headings, paragraphs, links, images, tables, lists, metadata, prices, dates.	✓
Grouped mode	Card-like grouped extraction and relationship preservation.	✓
CSV generation	Correct field order and export settings.	✓
Private IP logic	Local and private IP ranges are blocked.	✓

Reported Local Result

Core extraction functions are validated with controlled sample HTML rather than unstable live pages.

Strengths of the Project

Engineering Strengths

- ▶ Clean client-server separation.
- ▶ Modular extraction functions.
- ▶ Reusable CSV/JSON export logic.
- ▶ Backend tests for core functions.
- ▶ Clear development path for extensions.

Security Strengths

- ▶ Explicit SSRF mitigation.
- ▶ DNS and private IP validation.
- ▶ Manual redirect validation.
- ▶ Timeout and size controls.
- ▶ Non-HTML response rejection.

Current Limitations

Limitation	Professional Interpretation
Single-page extraction	The current version is controlled and predictable, but does not crawl entire websites.
JavaScript-rendered pages	Cheerio parses returned HTML only; content generated after browser execution may be incomplete.
Heuristic grouped extraction	Grouping works through structural patterns, not full semantic understanding.
Pattern-based prices and dates	Unusual formats may be missed or partially detected.
No authentication system	The current project is public-page focused and does not store user accounts or private data.
Website blocking	Some websites may reject automated requests.

Defense Position

These limitations are acceptable for the project scope because SmartFetch prioritizes safe public-page extraction and explainable architecture.

Future Security and Architecture Improvements

Security Improvements

- ▶ Rate limiting per IP/session.
- ▶ Authentication for saved jobs.
- ▶ Audit logging for submitted URLs.
- ▶ Stronger allow-list or deny-list policy.
- ▶ Container isolation for fetching logic.

Product Improvements

- ▶ Custom CSS selectors.
- ▶ Multi-page pagination extraction.
- ▶ Excel export.
- ▶ Saved extraction templates.
- ▶ Better JavaScript-rendered page support.

Conclusion

Final Summary

SmartFetch converts public webpage content into clean, previewable, and exportable data while securing backend-side URL fetching.

- ▶ The system separates validation, fetching, parsing, extraction, cleaning, and export.
- ▶ SSRF is addressed through URL, DNS, IP, redirect, timeout, and content controls.
- ▶ Tests validate core extraction and security-related helper functions.
- ▶ The architecture is clear for defense and extensible for future versions.

Closing Sentence

SmartFetch demonstrates secure software engineering applied to practical web data extraction.

Thank You

Questions and Discussion

SmartFetch Security Project Defense